Scaling Book — Section 10 Exercises

Rahul Bir

q1

part 1 (computing averages of each shard):

```
import jax
import jax.numpy as jnp
Auto = jax.sharding.AxisType.Auto
import functools
import numpy as np
# jax.config.update('jax_num_cpu_devices', 8)
```

```
# method 1: implemented with jax.jit
mesh = jax.make_mesh((4, 2), ('X', 'Y'), (Auto, Auto))
jax.set_mesh(mesh)

A = jnp.arange(32 * 64).reshape((32, 64))
A = jax.device_put(A, jax.P('X', 'Y'))


@functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
def shard_avg(A):
    r, c = jnp.shape(A)
    r_local, c_local = r // 4, c // 2
    return jnp.array([
        jnp.mean(A[r_local * i:r_local * i + r_local,
                   c_local * j:c_local * j + c_local],
                 axis=None)
        for i in range(4)
        for j in range(2)
    ]).reshape((4, 2))

with jax.profiler.trace("/kaggle/working/q1_p1_tensorboard_jit"):
    out_jit = shard_avg(A).block_until_ready()

print(out_jit)
```

```
# method 2: implemented with shard_map
mesh = jax.make_mesh((4, 2), ('X', 'Y'))
jax.set_mesh(mesh)

A = jnp.arange(32 * 64).reshape((32, 64))
A = jax.device_put(A, jax.P('X', 'Y'))

@jax.jit
@jax.shard_map(in_specs=jax.P('X', 'Y'), out_specs=jax.P('X', 'Y'))
def shard_avg_shm(A):
```

```
    return jnp.array([[jnp.mean(A)]])

with jax.profiler.trace("/kaggle/working/q1_p1_tensorboard_shm"):
    out_shm = shard_avg_shm(A).block_until_ready()

print(out_shm)
```

```
print(jnp.dtype(out_jit))
print(jnp.dtype(out_shm))
np.testing.assert_allclose(out_jit, out_shm)
```

The jit version takes around 65.7 micro seconds to complete whereas the shardmap version takes only 651.3 nano seconds. I was very surprised by the orders of magnitude difference! Another interesting thing is that the jit version added many communication primitives even though it needed to only run 1 fusion (which the shard map version does).

part 2 (implementing roll - x for each shard):

```
mesh = jax.make_mesh((4, 2), ('X', 'Y'))
jax.set_mesh(mesh)

A = jnp.arange(32 * 64).reshape((32, 64))
A = jax.device_put(A, jax.P('X', 'Y'))


def roll_rows_sub(x, shift):
    return jax.jit(jax.shard_map(
        lambda x: jnp.roll(x, shift=shift, axis=0) - x,
        in_specs=jax.P('X', 'Y'),
        out_specs=jax.P('X', 'Y')
    ))(x)

out = roll_rows_sub(A, 2)
print(out)

out = roll_rows_sub(A, 0)
ref = jnp.zeros((32, 64), dtype=jnp.int32, device=jax.NamedSharding(mesh, jax.P('X', 'Y')))
np.testing.assert_array_equal(out, ref)
```

q2

part 1:

```
"""
want to avoid materializing [S, D, F] array (very large!)

instead:

1) rearrange [S, D] sequence matrix into shape [E, S, D].
(Note: We need the array to be size [E, S, D] because normal jax arrays can't be ragged
 even though each expert has <= S tokens routed to it)

2) do matmul with W
```

```
11
12 3) index out based on correct experts
13 """
14
15 def moe_local(A, W, B):
16     return jnp.einsum('edf,esd->esf',
17                       W,
18                       jnp.expand_dims(A, 0)
19                      )[B, jnp.arange(S)]
```

part 2:

```
1  S = 8
2  D = 4
3  F = 32
4  E = 4
5
6  Auto = jax.sharding.AxisType.Auto
7  mesh = jax.make_mesh((1,), ('X',), (Auto,))
8  jax.set_mesh(mesh)
9
10 @jax.jit
11 def init_arrays(key1, key2, key3):
12     W = jax.random.normal(key1,
13                           (E, D, F),
14                           dtype=jnp.float32)
15
16     A = jax.random.normal(key2,
17                           (S, D),
18                           dtype=jnp.float32)
19
20     B = jax.random.randint(key3,
21                            (S,),
22                            dtype=jnp.int32,
23                            minval=0,
24                            maxval=E)
25
26     W = jax.device_put(W, jax.P('X', None, None))
27     A = jax.device_put(A, jax.P('X', None))
28     B = jax.device_put(B, jax.P('X'))
29
30     return W, A, B
31
32
33 key1 = jax.random.key(42)
34 key2, _ = jax.random.split(key1)
35 key3, _ = jax.random.split(key2)
36
37
38 moe_jit = jax.jit(moe_local)
39 W, A, B = init_arrays(key1, key2, key3)
40
41 with jax.profiler.trace("/kaggle/working/q2_p2_tensorboard_naive_jit"):
42     out_jit = moe_jit(A, W, B).block_until_ready()
43
44 print('output out_jit:')
45 print(out_jit)
```

part 3:

```
import numpy as np

"""
pass 1 ideas:
AG_X(W[E@X, D, F]) = W[E, D, F]
W[E, D, F] @_D A[E, S@X, D] => [E, S@X, D]
Index correct values based on B[S@X]
"""

@jax.jit
@jax.shard_map(in_specs=(jax.P('X', None), jax.P('X', None, None), jax.P('X')),
    out_specs=jax.P('X', None))
def moe_shm(A, W, B):
    W = jax.lax.all_gather(W, 'X', tiled=True)
    return jnp.einsum('edf,esd->esf',
                      W,
                      jnp.expand_dims(A, 0)
                     )[B, jnp.arange(S // 8)]


out_shm = moe_shm(A, W, B)
out_jit = moe_jit(A, W, B)
print(jax.typeof(out_shm))

np.testing.assert_allclose(out_jit, out_shm)


"""
inefficiencies:
1) need to materialize W[E, D, F] array on each device
2) wasteful computation on "padded" [E, S, D] array (need to make it ragged)
"""
```

```
"""
pass 2:

main ideas:
1) create ragged array to minimize empty padding flops
2) get rid of big AG and replace with cheaper AllToAlls
"""

@jax.jit
@jax.shard_map(in_specs=(jax.P('X', None), jax.P('X', None, None), jax.P('X')),
    out_specs=jax.P('X', None))
def moe_shm_chunked(A, W, B):
    sorted_B_inds = jnp.argsort(B)
    sorted_B = B[sorted_B_inds]
    expert_sorted_A = A[sorted_B_inds]
    E = W.shape[0] * jax.lax.axis_size('X')
    sizes = jnp.bincount(B, minlength=E, length=E)
    global_max_expert_size = jnp.max(jax.lax.pmax(sizes, 'X'))
    start_inds = jnp.cumsum(sizes) - sizes


    CHUNK_SIZE = 8
    def get_chunk(i):
        inds = (CHUNK_SIZE * i) + start_inds[:, None] + jnp.arange(CHUNK_SIZE)[
```

```
            None, :]
        chunk = expert_sorted_A[jnp.ravel(inds)].reshape((E, CHUNK_SIZE, A.shape
            [1]))
        end_inds = jnp.append(jax.lax.dynamic_slice_in_dim(start_inds, 1, E-1),
            jnp.array([A.shape[0]]))
        valid = inds < end_inds[:, None]
        return chunk, valid, inds


    def f(i, out):
        chunk, valid, inds = get_chunk(i)
        chunk = jax.lax.all_to_all(chunk, 'X', split_axis=0, concat_axis=1, tiled=
            True)
        chunk = jnp.einsum('ecd,edf->ecf', chunk, W)
        chunk = jax.lax.all_to_all(chunk, 'X', split_axis=1, concat_axis=0, tiled=
            True)
        # setting values later for the update is nondeterministic, so write
            invalid inds to a padding row instead
        masked_inds = jnp.where(valid, inds, -1)
        flattened_inds = jnp.ravel(masked_inds)
        flattened_chunk = chunk.reshape(E * CHUNK_SIZE, W.shape[2])
        flattened_valid = jnp.ravel(valid)[:, None]
        update = jnp.where(flattened_valid, flattened_chunk, 0)
        new_out = out.at[flattened_inds].set(update)
        return new_out

    # add padding rows so writes on padding don't overwrite real values
    out = jnp.zeros((A.shape[0] + 1, W.shape[2]))
    out = jax.lax.pcast(out, 'X', to='varying')
    out = jax.lax.fori_loop(0, (global_max_expert_size + CHUNK_SIZE - 1)  //
        CHUNK_SIZE, f, out)

    return out[jnp.argsort(sorted_B_inds)]
```

```
# run comparison test for thoroughness
Auto = jax.sharding.AxisType.Auto
Explicit = jax.sharding.AxisType.Explicit


def run_test(S, D, F, E, seed):
    with jax.set_mesh(jax.make_mesh((8,), ('X'), (Auto,))):
        key1 = jax.random.key(seed)
        key2, key3 = jax.random.split(key1)
        W = 10 * jax.random.normal(key1,
                           (E, D, F),
                           dtype=jnp.float32)
        A = 10 * jax.random.normal(key2,
                               (S, D),
                               dtype=jnp.float32)
        B = jax.random.randint(key3,
                               (S,),
                               dtype=jnp.int32,
                               minval=0,
                               maxval=E)
        W = jax.device_put(W, jax.P('X', None, None))
        A = jax.device_put(A, jax.P('X', None))
        B = jax.device_put(B, jax.P('X'))

        moe_jit = jax.jit(moe_local)
```

```
        with jax.profiler.trace("/kaggle/working/q2_p2_tensorboard_naive_jit"):
            out_jit = moe_jit(A, W, B).block_until_ready()

    with jax.set_mesh(jax.make_mesh((8,), ('X',), (Explicit,))):
        key1 = jax.random.key(seed)
        key2, key3 = jax.random.split(key1)
        W = 10 * jax.random.normal(key1,
                    (E, D, F),
                    dtype=jnp.float32,
                    out_sharding=jax.P('X', None, None))
        A = 10 * jax.random.normal(key2,
                            (S, D),
                            dtype=jnp.float32,
                            out_sharding=jax.P('X', None))
        B = jax.random.randint(key3,
                             (S,),
                             dtype=jnp.int32,
                             minval=0,
                             maxval=E,
                             out_sharding=jax.P('X'))

        with jax.profiler.trace("/kaggle/working/q2_p3_tensorboard_shm_chunked"):
            out_shm_chunked = moe_shm_chunked(A, W, B).block_until_ready()


    np.testing.assert_allclose(out_shm_chunked, out_jit, rtol=1e-5, atol=1e-5)
    print("TEST PASSED")



S = 512
D = 4096
F = 4096*4
E = 8

"""
With 16 experts, and 512 tokens.

Want to minimize padding. In best case scenario
tokens are balanced fully across each expert,
so each expert gets 512/16 = 32 tokens routed
to them. => choosing a CS=8 seems reasonable.
"""

run_test(S, D, F, E, 42)
```

The naive jit kernel takes 2.665 ms, whereas the chunked moe shardmap version I wrote above takes 1.669 ms! This is under the S, D, F, E values above on a v5e-8 cluster.

The roofline for this is $2 \cdot S \cdot D \cdot F/(8 \cdot C) = 0.08$ ms.

part 4

```
"""
route to (k) experts and average the result

W[E@X, D, F]
A[S@X, D]
```

```
B[S@X, k]

qs:

- how to fill chunks? -> need to map the same token to multiple experts


core idea:

reconstruct into S*k out, keep logic the same, add extra k dimension when writing back?
"""
@jax.jit
def moe_k_experts_naive(A, W, B):
    """
    W[E@X, D, F]
    A[S@X, D]
    B[S@X, k]
    """
    out = []
    for s_i in range(A.shape[0]):
        token = A[s_i] # [D]
        experts = B[s_i] # [k]
        selected_W = W[experts] # [k, D, F]
        out.append(jnp.einsum('kdf,d->kf', selected_W, token).mean(axis=0))

    return jnp.stack(out)


@jax.jit
@jax.shard_map(in_specs=(jax.P('X', None), jax.P('X', None, None), jax.P('X', None)), out_specs=jax.P('X', None))
def moe_k_experts_shm(A, W, B):
    """
    B[S@X, k]

    b_flat = B[S_x*k,]
    b_flat_sorted = sort(B[S_x*k,]) => B[S_x*k,]
    A_inds = jnp.arange

    """

    # jax.debug.print('ORIGINAL B: \n {b}', b=B, ordered=True)
    flattened_B = B.flatten()
    # jax.debug.print('FLATTENED B: {b}', b=flattened_B, ordered=True)
    sorted_flat_B_inds = jnp.argsort(flattened_B)
    sorted_flat_B = flattened_B[sorted_flat_B_inds]
    # jax.debug.print('SORTED FLAT B inds: \n {b}', b=sorted_flat_B_inds, ordered=True)
    # jax.debug.print('SORTED FLAT B: \n {b}', b=sorted_flat_B, ordered=True)
    rep_flat_A_inds = jnp.repeat(
        jnp.arange(A.shape[0])[:, None],
        B.shape[1],
        axis=-1,
        total_repeat_length=B.shape[1]
    ).flatten()
    # jax.debug.print('repeated flat A inds: \n {b}', b=rep_flat_A_inds, ordered=True)
    sorted_rep_flat_A_inds = rep_flat_A_inds[sorted_flat_B_inds]
```

```
    # jax.debug.print('sorted repeated flat A inds: \n {b}', b=
        sorted_rep_flat_A_inds, ordered=True)
    E = W.shape[0] * jax.lax.axis_size('X')
    sizes = jnp.bincount(flattened_B, minlength=E, length=E)
    # jax.debug.print('sizes: \n {s}', s=sizes, ordered=True)
    global_max_expert_size = jnp.max(jax.lax.pmax(sizes, 'X'))
    start_inds = jnp.cumsum(sizes) - sizes
    # jax.debug.print('start_inds: \n {s}', s=start_inds, ordered=True)

    CHUNK_SIZE = 16
    def get_chunk(i):
        """
        inds is relative to the sorted_A_inds
        """
        # jax.debug.print('----------', ordered=True)
        # jax.debug.print('getting chunk {i}', i=i, ordered=True)
        # jax.debug.print('original A for reference: \n {a}', a=A, ordered=True)
        # inds[E, CS]
        inds = (CHUNK_SIZE * i) + start_inds[:, None] + jnp.arange(CHUNK_SIZE)[
            None, :]
        # jax.debug.print('inds to get from sortedA: \n {a}', a=inds, ordered=True
            )
        # jax.debug.print('corresponding original A inds: \n {i}', i=
            sorted_rep_flat_A_inds[inds.flatten()].reshape((E, CHUNK_SIZE)),
            ordered=True)
        # chunk[]
        chunk = A[sorted_rep_flat_A_inds[inds.flatten()]].reshape((E, CHUNK_SIZE,
            A.shape[1]))
        # jax.debug.print('CHUNK: \n {c}', c=chunk, ordered=True)
        end_inds = jnp.append(jax.lax.dynamic_slice_in_dim(start_inds, 1, E-1),
            jnp.array([A.shape[0] * B.shape[1]]))
        valid = inds < end_inds[:, None]
        # jax.debug.print('valid mask: \n {c}', c=valid, ordered=True)
        # jax.debug.print('----------', ordered=True)
        return chunk, valid, inds


    def f(i, out):
        # jax.debug.print('----- STARTING F {i}', i=i, ordered=True)
        chunk, valid, inds = get_chunk(i)
        # jax.debug.print('chunk received: \n {c}', c=chunk, ordered=True)
        # jax.debug.print('valid received: \n {c}', c=valid, ordered=True)
        # jax.debug.print('inds received: \n {c}', c=inds, ordered=True)
        chunk = jax.lax.all_to_all(chunk, 'X', split_axis=0, concat_axis=1, tiled=
            True)
        chunk = jnp.einsum('ecd,edf->ecf', chunk, W)
        # jax.debug.print('after einsum chunk: \n {c}', c=chunk, ordered=True)
        chunk = jax.lax.all_to_all(chunk, 'X', split_axis=1, concat_axis=0, tiled=
            True)
        # setting values later for the update is nondeterministic, so write
            invalid inds to a padding row instead
        masked_inds = jnp.where(valid, inds, -1)
        # jax.debug.print('masked inds: \n {c}', c=masked_inds, ordered=True)
        flattened_inds = jnp.ravel(masked_inds)
        flattened_chunk = chunk.reshape(E * CHUNK_SIZE, W.shape[2])
        flattened_valid = jnp.ravel(valid)[:, None]
        update = jnp.where(flattened_valid, flattened_chunk, 0)
        new_out = out.at[flattened_inds].set(update)
        # jax.debug.print('new out: \n {n}', n=new_out, ordered=True)
```

```
        return new_out


    out = jnp.zeros((A.shape[0] * B.shape[1] + 1, W.shape[2]))
    # out = jax.lax.pcast(out, 'X', to='varying')
    out = jax.lax.pvary(out, 'X')
    out = jax.lax.fori_loop(0, (global_max_expert_size + CHUNK_SIZE - 1)  //
        CHUNK_SIZE, f, out)
    out = out[jnp.argsort(sorted_flat_B_inds)].reshape((A.shape[0], B.shape[1], W.
        shape[2])).mean(axis=1)

    return out


S = 512
D = 4096
F = 4096*4
E = 8
k = 4
seed = 42

"""
512*4 = 2048 tokens to route + process
2048/8 = 256
=> setting a CS = 16 seems reasonable
"""

Explicit = jax.sharding.AxisType.Explicit
Auto = jax.sharding.AxisType.Auto

def run_test(S, D, F, E, k, seed):
    def generate_B(seed, k, E, S):
        out = []
        key = jax.random.key(seed)
        for _ in range(S):
            key, subkey = jax.random.split(key)
            out.append(jax.random.choice(subkey, E, (k,), replace=False))
        return jnp.stack(out)

    with jax.set_mesh(jax.make_mesh((8,), ('X',), (Explicit,))):
        key1 = jax.random.key(seed)
        key2, key3 = jax.random.split(key1)
        W = 10 * jax.random.normal(key1,
                      (E, D, F),
                      dtype=jnp.float32,
                      out_sharding=jax.P('X', None, None))
        A = 10 * jax.random.normal(key2,
                              (S, D),
                              dtype=jnp.float32,
                              out_sharding=jax.P('X', None))
        B = generate_B(seed, k, E, S)
        B = jax.device_put(B, jax.P('X', None))

        out_shm_chunked = moe_k_experts_shm(A, W, B).block_until_ready()

    with jax.set_mesh(jax.make_mesh((8,), ('X',), (Auto,))):
        key1 = jax.random.key(seed)
        key2, key3 = jax.random.split(key1)
        W = 10 * jax.random.normal(key1,
```

```
                        (E, D, F),
                        dtype=jnp.float32)
        A = 10 * jax.random.normal(key2,
                                   (S, D),
                                   dtype=jnp.float32)
        B = generate_B(seed, k, E, S)
        W = jax.device_put(W, jax.P('X', None, None))
        B = jax.device_put(B, jax.P('X', None))
        A = jax.device_put(A, jax.P('X', None))

        out_naive = moe_k_experts_naive(A, W, B).block_until_ready()


    np.testing.assert_allclose(out_shm_chunked, out_naive, rtol=1e-5, atol=1e-5)
    print('TEST PASSED')


run_test(S, D, F, E, k, seed)
```

q3

```
import jax
import jax.numpy as jnp
import functools
import numpy as np
jax.config.update('jax_num_cpu_devices', 8)
```

part 1 (implementing AllReduce collective matmul):

```
"""
A[B@X, D@Y] *_D W[D@Y, F] => Out[B@X, F]

Naive impl:
A[B@X, D@Y] *_D W[D@Y, F] => Out[B@X, F]{U_Y}
AR_Y(Out[B@X, F]{U_Y}) => Out[B@X, F]

Can't overlap comm + comp...


Idea:

Take simple 2x2 mesh case:

A = [[A1  A2],
     [A3  A4]]

W = [[B1],
     [B2]]

Out = [[O1],
       [O2]]

Doing AW = Out

O1 = A1*B1 + A2*B2 (hence all reduce)

```

```
want to tile this further to try to overlap comm + comp

let B1 = [b_11  b_12]
    B2 = [b_21  b_22]


then:

O1 = A1 * [b_11  b_12] + A2 * [b_21  b_22] = [A1*b_11  A1*b_12] + [A2*b_21  A2*b_22] =
     [(A1*b_11 + A2*b_21)   (A1*b_12 + A2*b_22)]

core algorithm idea:

chunk the F dimension of W up and loop over the chunks on each device:
- Do local matmul with local A shard
- AllReduce that small chunk
- Add to an array and concat them
"""

@jax.jit
@jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X', None))
def collective_matmul_all_reduce(A, W):
    CHUNK_SIZE = 1024
    def f(i, out):
        chunk_W = jax.lax.dynamic_slice_in_dim(W, CHUNK_SIZE * i, CHUNK_SIZE, axis=-1)
        out_chunk = jnp.einsum('bd,dc->bc', A, chunk_W)
        out_chunk = jax.lax.psum(out_chunk, 'Y')
        out = jax.lax.dynamic_update_slice_in_dim(out, out_chunk, CHUNK_SIZE * i, axis=-1)
        return out

    out = jnp.zeros((A.shape[0], W.shape[1]), dtype=jnp.int32)
    out = jax.lax.pcast(out, 'X', to='varying')
    out = jax.lax.fori_loop(0, (W.shape[1] + CHUNK_SIZE - 1) // CHUNK_SIZE, f, out, unroll=True)

    return out


@jax.jit
def matmul(A, W):
  return jnp.einsum('bd,df->bf', A, W, out_sharding=jax.P('X', None))


Explicit = jax.sharding.AxisType.Explicit
mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
jax.set_mesh(mesh)

B = 1024
D = 2048
F = 8192

A = jnp.arange(B * D).reshape((B, D))
W = jnp.arange(D * F).reshape((D, F))

A = jax.device_put(A, jax.P('X', 'Y'))
```

```
W = jax.device_put(W, jax.P('Y', None))

with jax.profiler.trace("/kaggle/working/q3_p1_tensorboard_jit"):
    out_baseline = matmul(A, W).block_until_ready()

"""
Set chunk size large enough to be out of latency bound regime
"""

with jax.profiler.trace("/kaggle/working/q3_p1_tensorboard_collective_matmul"):
    out_coll_matmul = collective_matmul_all_reduce(A, W).block_until_ready()

np.testing.assert_allclose(out_coll_matmul, out_baseline)
```

The jit version takes around 517 microseconds whereas the collective matmul version takes around 550 microseconds. Looking at the profile, this seems to be because the xla compiler isn't overlapping the chunked matmul fusions and ARs together. This ends up taking the same time plus the additional overhead of launching many smaller ops.

part 2 (implementing a RS collective matmul):

```
"""
Tmp[B@X, F@Y] *_F W2[F@Y, D] => Out[B@X, D@Y]

Naive impl:
Tmp[B@X, F@Y] *_F W2[F@Y, D] => Out[B@X, D]{U_Y}
RS_Y,D(Out[B@X, D]{U_Y}) => Out[B@X, D@Y]
(again can't overlap comm and comp)

Idea:

Take simple 2x2 mesh case:

A = [[A11  A12],
     [A21  A22]]

B = [[B11  B12],
     [B21  B22]]

Out = [[O11  O12],
       [O21  O22]]

Note: d# notation refers to what device that shard is on

d1(O11) = d1(A11*B11) + d2(A12*B21)
d2(O12) = d1(A11*B12) + d2(A12*B22)
d3(O21) = d3(A21*B11) + d4(A22*B21)
d4(O22) = d3(A21*B12) + d4(A22*B22)



observations:
- each A shard only needs to multiply with it's corresponding y axis index B shard
    as it and the result permute through


core idea:
```

```
do local matmuls where chunk from W matrix corresponds to what y axis and shift the accumulator.
note: we must start by doing the local matmul for the next device's shard to have one less comm than comp.
      this way, every comm is overlapped by a comp

we get on first step (only looking at first row of output, since the pattern can be used for other rows as well):
O11 = d2(A12*B21)
O12 = d1(A11*B12)

O12 needs d2(A12*B22)
O11 needs d1(A11*B11)

move accumulator back to corresponding shard, and update accum. we needed 2 local matmuls but only 1 comp.
"""

@jax.jit
@jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X', 'Y'))
def collective_matmul_reduce_scatter(A, W):
    axis_size = jax.lax.axis_size('Y')
    idx = jax.lax.axis_index('Y')
    chunk_size = W.shape[1] // axis_size

    def f(i, acc):
        W_chunk = jax.lax.dynamic_slice_in_dim(
            W,
            chunk_size * ((idx + 1 + i) % axis_size),
            chunk_size,
            axis=1
        )
        local_res = jnp.einsum('bf,fc->bc', A, W_chunk)
        acc += local_res

        return jax.lax.ppermute(
            acc,
            'Y',
            [(i, (i-1)%axis_size) for i in range(axis_size)]
        )


    out = jnp.zeros((A.shape[0], chunk_size), dtype=jnp.int32)
    out = jax.lax.pcast(out, ('X', 'Y'), to='varying')
    # out = jax.lax.pvary(out, ('X', 'Y'))
    last_acc = jax.lax.fori_loop(0, axis_size - 1, f, out, unroll=True)

    # last local matmul doesn't require another ppermute
    last_W_chunk = jax.lax.dynamic_slice_in_dim(
        W,
        chunk_size * ((idx + axis_size) % axis_size),
        chunk_size,
        axis=1
    )
    last_acc += jnp.einsum('bf,fc->bc', A, last_W_chunk)
    return last_acc


```

```python
@jax.jit
def matmul(A, W):
    return jnp.einsum(
        'bf,fd->bd',
        A,
        W,
        out_sharding=jax.P('X', 'Y')
    )


Explicit = jax.sharding.AxisType.Explicit
mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
jax.set_mesh(mesh)

B = 1024
D = 2048
F = 8192

A = jnp.arange(B*F).reshape((B, F))
W = jnp.arange(F*D).reshape((F, D))

A = jax.device_put(A, jax.P('X', 'Y'))
W = jax.device_put(W, jax.P('Y', None))

with jax.profiler.trace("/kaggle/working/q3_p2_tensorboard_collective_matmul"):
    out = collective_matmul_reduce_scatter(A, W).block_until_ready()

with jax.profiler.trace("/kaggle/working/q3_p2_tensorboard_jit"):
    out_baseline = matmul(A, W).block_until_ready()

np.testing.assert_array_equal(out, out_baseline)
print('TEST PASSED')
```

The naive jit version takes around 300us whereas the collective all reduce version takes 280us.

part 3 (putting them together into an end-to-end Transformer block):

```python
@jax.jit
@jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P(None, 'Y')), out_specs=jax.P('X', 'Y'))
def collective_matmul_all_gather(A, W):
    """
    core idea:
    rotate A chunks and index out corresponding W chunk. Keep running accumulator and final result is already on the device
    """
    axis_size = jax.lax.axis_size('Y')
    idx = jax.lax.axis_index('Y')
    chunk_size = A.shape[1]

    def f(i, carry):
        out, A_chunk = carry
        W_chunk = jax.lax.dynamic_slice_in_dim(W, chunk_size * ((idx - i) % axis_size), chunk_size)
        out += jnp.einsum('bd,df->bf', A_chunk, W_chunk)
        A_chunk = jax.lax.ppermute(A_chunk, 'Y', [(i, (i+1)%axis_size) for i in range(axis_size)])
        return out, A_chunk
```

```
    out = jnp.zeros((A.shape[0], W.shape[1]), dtype=jnp.int32)
    out = jax.lax.pcast(out, ('X', 'Y'), to='varying')
    # out = jax.lax.pvary(out, ('X', 'Y'))
    out, last_A_chunk = jax.lax.fori_loop(0, axis_size - 1, f, (out, A), unroll=
        True)

    # do last local matmul independently so we don't do extra comm
    last_W_chunk = jax.lax.dynamic_slice_in_dim(W, chunk_size * ((idx - (axis_size
         - 1)) % axis_size), chunk_size)
    out += jnp.einsum('bd,df->bf', last_A_chunk, last_W_chunk)
    return out


@functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
def matmul(A, W):
    return A @ W


Explicit = jax.sharding.AxisType.Explicit
mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
jax.set_mesh(mesh)

B = 1024
D = 2048
F = 8192

A = jnp.arange(B*D).reshape((B, D))
W = jnp.arange(F*D).reshape((D, F))

A = jax.device_put(A, jax.P('X', 'Y'))
W = jax.device_put(W, jax.P(None, 'Y'))

with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_ag_collective_matmul"):
    out = collective_matmul_all_gather(A, W)

with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_ag_jit"):
    out_baseline = matmul(A, W)

np.testing.assert_array_equal(out, out_baseline)
print("TEST PASSED")
```

```
"""
In[B@X, D@Y] *_D W_in[D, F@Y] *_F W_out[F@Y, D]
"""

@jax.jit
def collective_mlp_block(In, W1, W2):
    tmp = collective_matmul_all_gather(In, W1)
    return collective_matmul_reduce_scatter(tmp, W2)


@functools.partial(jax.jit, out_shardings=jax.P('X', 'Y'))
def mlp_block(In, W1, W2):
    tmp = jnp.einsum('bd,df->bf', In, W1, out_sharding=jax.P('X', 'Y'))
    return jnp.einsum('bf,fd->bd', tmp, W2, out_sharding=jax.P('X', 'Y'))


B = 1024
```

```
D = 2048
F = 8192

In = jnp.arange(B*D).reshape((B, D))
W1 = jnp.arange(D*F).reshape((D, F))
W2 = jnp.arange(F*D).reshape((F, D))

In = jax.device_put(In, jax.P('X', 'Y'))
W1 = jax.device_put(W1, jax.P(None, 'Y'))
W2 = jax.device_put(W2, jax.P('Y', None))

with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_collective_mlp"):
    out = collective_mlp_block(In, W1, W2)

with jax.profiler.trace("/kaggle/working/q3_p3_tensorboard_mlp_jit"):
    out_baseline = mlp_block(In, W1, W2)

np.testing.assert_array_equal(out, out_baseline)
print('TEST PASSED')
```

The collective matmul version took 510us whereas the jit version took 585us.

q4

```
import jax
import jax.numpy as jnp
import functools
import numpy as np
# jax.config.update('jax_num_cpu_devices', 8)
```

```
"""
core change to make it bidirectional:
split the one big all to all into 2 smaller alltoalls and pray that xla compiles
    bidirectionally
"""


@jax.jit
@jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X',
    None))
def collective_matmul_all_reduce_bidir(A, W):
    CHUNK_SIZE = 1024
    def f(i, out):
        chunk_W = jax.lax.dynamic_slice_in_dim(W, CHUNK_SIZE * i, CHUNK_SIZE, axis
            =-1)
        local_res = jnp.einsum('bd,dc->bc', A, chunk_W)
        out_subchunk_1 = jax.lax.psum(
            jax.lax.dynamic_slice_in_dim(local_res, 0, CHUNK_SIZE // 2, axis=-1),
            'Y'
        )
        out_subchunk_2 = jax.lax.psum(
            jax.lax.dynamic_slice_in_dim(local_res, CHUNK_SIZE // 2, CHUNK_SIZE //
                2, axis=-1),
            'Y'
        )
        out = jax.lax.dynamic_update_slice_in_dim(
            out,
```

```
            jnp.hstack([out_subchunk_1, out_subchunk_2]),
            CHUNK_SIZE * i,
            axis=-1
        )
        return out

    out = jnp.zeros((A.shape[0], W.shape[1]), dtype=jnp.int32)
    out = jax.lax.pcast(out, 'X', to='varying')
    # out = jax.lax.pvary(out, 'X')
    out = jax.lax.fori_loop(0, (W.shape[1] + CHUNK_SIZE - 1) // CHUNK_SIZE, f, out
        , unroll=True)

    return out


Explicit = jax.sharding.AxisType.Explicit
mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
jax.set_mesh(mesh)

B = 1024
D = 2048
F = 8192

A = jnp.arange(B * D).reshape((B, D))
W = jnp.arange(D * F).reshape((D, F))

A = jax.device_put(A, jax.P('X', 'Y'))
W = jax.device_put(W, jax.P('Y', None))

with jax.profiler.trace("/kaggle/working/q4_tensorboard_unidir_ar_matmul"):
    out_coll_matmul_unidir = collective_matmul_all_reduce(A, W).block_until_ready
        ()

"""
Set chunk size large enough to be out of latency bound regime
"""

with jax.profiler.trace("/kaggle/working/q4_tensorboard_bidir_ar_matmul"):
    out_coll_matmul_bidir = collective_matmul_all_reduce_bidir(A, W).
        block_until_ready()

np.testing.assert_array_equal(out_coll_matmul_unidir, out_coll_matmul_bidir)
print("TEST PASSED")
```

The unidirectional AllReduce collective matmul took 561us whereas the bidirectional AllReduce collective matmul took 590us. Looking at the trace, the xla compiler doesn't seem to fuse the two AllReduce's and do them bidirectionally.

```
"""
idea:

split the A chunk into two halves (split by rows), move on left and one right
and get the corresponding slices of the W matrix as you process them.

indexing changes depending on the direction you permute chunk but reasoning is
    similar to
unidirectional case.
"""
```

```python


@jax.jit
@jax.shard_map(in_specs=(jax.P('X', 'Y'), jax.P('Y', None)), out_specs=jax.P('X', 'Y'))
def collective_matmul_reduce_scatter_bidir(A, W):
    axis_size = jax.lax.axis_size('Y')
    idx = jax.lax.axis_index('Y')
    chunk_size = W.shape[1] // axis_size
    top_A_slice = jax.lax.dynamic_slice_in_dim(A, 0, A.shape[0] // 2)
    bottom_A_slice = jax.lax.dynamic_slice_in_dim(A, A.shape[0] // 2, A.shape[0] - (A.shape[0] // 2))

    def f(i, carry):
        top_acc, bottom_acc = carry
        top_W_chunk = jax.lax.dynamic_slice_in_dim(
            W,
            chunk_size * ((idx + 1 + i) % axis_size),
            chunk_size,
            axis=1
        )
        top_local_res = jnp.einsum('bf,fc->bc', top_A_slice, top_W_chunk)
        top_acc += top_local_res
        top_acc = jax.lax.ppermute(
            top_acc,
            'Y',
            [(i, (i-1)%axis_size) for i in range(axis_size)]
        )

        bottom_W_chunk = jax.lax.dynamic_slice_in_dim(
            W,
            chunk_size * ((idx - 1 - i) % axis_size),
            chunk_size,
            axis=1
        )
        bottom_local_res = jnp.einsum('bf,fc->bc', bottom_A_slice, bottom_W_chunk)
        bottom_acc += bottom_local_res
        bottom_acc = jax.lax.ppermute(
            bottom_acc,
            'Y',
            [(i, (i+1)%axis_size) for i in range(axis_size)]
        )

        return top_acc, bottom_acc


    top_acc = jnp.zeros((top_A_slice.shape[0], chunk_size), dtype=jnp.int32)
    top_acc = jax.lax.pcast(top_acc, ('X', 'Y'), to='varying')
    # top_acc = jax.lax.pvary(top_acc, ('X', 'Y'))
    bottom_acc = jnp.zeros((bottom_A_slice.shape[0], chunk_size), dtype=jnp.int32)
    bottom_acc = jax.lax.pcast(bottom_acc, ('X', 'Y'), to='varying')
    # bottom_acc = jax.lax.pvary(bottom_acc, ('X', 'Y'))

    top_acc, bottom_acc = jax.lax.fori_loop(0, axis_size - 1, f, (top_acc, bottom_acc), unroll=True)

    # last local matmul doesn't require another ppermute
    last_W_chunk = jax.lax.dynamic_slice_in_dim(
        W,
```

```
        chunk_size * ((idx + axis_size) % axis_size),
        chunk_size,
        axis=1
    )
    top_acc += jnp.einsum('bf,fc->bc', top_A_slice, last_W_chunk)
    bottom_acc += jnp.einsum('bf,fc->bc', bottom_A_slice, last_W_chunk)
    return jnp.vstack([top_acc, bottom_acc])


@jax.jit
def matmul(A, W):
    return jnp.einsum(
        'bf,fd->bd',
        A,
        W,
        out_sharding=jax.P('X', 'Y')
    )


Explicit = jax.sharding.AxisType.Explicit
mesh = jax.make_mesh((2, 4), ('X', 'Y'), (Explicit, Explicit))
jax.set_mesh(mesh)

B = 1024
D = 2048
F = 8192

A = jnp.arange(B*F).reshape((B, F))
W = jnp.arange(F*D).reshape((F, D))

A = jax.device_put(A, jax.P('X', 'Y'))
W = jax.device_put(W, jax.P('Y', None))

with jax.profiler.trace("/kaggle/working/q4_tensorboard_unidir_rs_matmul"):
    out_unidir = collective_matmul_reduce_scatter(A, W).block_until_ready()

with jax.profiler.trace("/kaggle/working/q4_tensorboard_bidir_rs_matmul"):
    out_bidir = collective_matmul_reduce_scatter_bidir(A, W).block_until_ready()

np.testing.assert_array_equal(out_unidir, out_bidir)
print('TEST PASSED')
```

The unidirectional ReduceScatter collective matmul took 280us whereas the bidirectional ReduceScatter collective matmul took 250us.